COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/04/26	**DATE SUBMITTED:**	09/08/26
NAME:	Villaruel, Prince Benjamin S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 **Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	main.cpp > main()
	1 #include <iostream>
	2 #include <cstdlib>
	3 #include <time.h>
	4 #include "sortingAlgo.h"
	5
	6 const int maxSize = 100;
	7
	8 void displayArray(int arr[]);
	9
	10 int main(){
	11
	12 //create an array with unsorted list with 100 maximum size
	13 int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
	14 // use time (0) as seed for random values
	15 srand(time(0));
	16 //populate the array with random values
	17 for(int i = 0; i < maxSize; i++){
	18 unsortedArr[i] = rand()% 100; // random values between 0 and 99
	19 }
	20 //display the randomly displayed array
	21 std::cout << "Unsorted Array: \n" << std::endl;
	22 displayArray(unsortedArr);
	23
	24 // std::cout << "\nBubble Sort: \n" << std::endl;
	25 // bubbleSort(unsortedArr, maxSize);
	26 // displayArray(unsortedArr);
	27
	28 // std::cout << "\nSelection Sort: \n" << std::endl;
	29 // selectionSort(unsortedArr, maxSize);
	30 // displayArray(unsortedArr);
	31
	32 // std::cout << "\nInsertion Sort: \n" << std::endl;
	33 // insertionSort(unsortedArr, maxSize);
	34 // displayArray(unsortedArr);
	35 }
	36
	37 void displayArray(int arr[]) {
	38
	39 for (int i = 0; i < maxSize; i++) {
	40 std::cout << arr[i] << " ";
	41 }
	42 std::cout << std::endl;
	43 }
	
	Unsorted Array:
	
	74 80 11 71 37 62 33 0 1 30 5 96 27 9 73 46 95 52 31 30 10 10 90 95 20 45 23 61 97 4 87 92 24 73 76 66 64 79 82 36 19 50 72 80
	8 54 24 73 12 80 75 67 90 38 52 53 15 38 43 25 64 74 13 22 30 62 54 27 77 77 77 99 23 68 11 0 79 17 10 29 92 96 52 59 64 72 85
	78 39 8 89 68 90 69 42 93 28 3 36 71

Observations	For the first task, I initialized an integer array with a maximum size of 100 elements in C++ and used srand(time(0)) alongside rand() % 100 to populate it with pseudo-random values ranging from 0 to 99. I created a custom displayArray function to iterate through and output the complete unsorted dataset to the console.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);

    std::cout << "\nBubble Sort: \n" << std::endl;
    bubbleSort(unsortedArr, maxSize);
    displayArray(unsortedArr);

    // std::cout << "\nSelection Sort: \n" << std::endl;
    // selectionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    // std::cout << "\nInsertion Sort: \n" << std::endl;
    // insertionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

sortingAlgo.h > routineSmallest<T>(T [], int, const int)

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include<cstddef>
#include<iostream>
template<typename T>
void bubbleSort(T arr[], size_t arrSize){
    //step 1 for i = 0 to N-1 repeat inner loop
    for(int i = 0; i < arrSize; i++){
        //step 2 inner loop that for K = i + 1 to N -I repeat
        for(int j = 0; j < arrSize - i - 1; j++){
            //step 3 swap if the arr[j]  > arr[i]
            if (arr[j] > arr[j+1]){
                //swap
                T temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 7> cd "c:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 7\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Unsorted Array:

40 82 22 73 60 31 24 77 19 46 71 75 3 95 0 65 96 60 42 76 83 51 73 40 67 10 71 42 20 74 76 76 57 97 68 75 29 63 46 99 90 86 95
24 70 69 43 85 89 74 32 10 98 96 13 66 74 99 44 24 70 57 19 64 17 97 47 78 93 34 31 23 68 10 92 97 46 23 25 96 1 62 56 24 97 54
 53 33 29 24 58 44 10 93 72 68 82 87 38 9

Bubble Sort:

0 1 3 9 10 10 10 10 13 17 19 19 20 22 23 23 24 24 24 24 24 25 29 29 31 31 32 33 34 38 40 40 42 42 43 44 44 46 46 46 47 51 53 54
 56 57 57 58 60 60 62 63 64 65 66 67 68 68 68 69 70 70 71 71 72 73 73 74 74 74 75 75 76 76 76 77 78 82 82 83 85 86 87 89 90 92
93 93 95 95 96 96 96 97 97 97 97 98 99 99
```

Observations

For the second task, in sortingAlgo.h, I implemented a templated bubble sort function using nested loops to swap adjacent elements, which has both a best

	case and worst case time complexity of O(n^2). The console output confirmed the random array was successfully sorted in ascending order.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	

main.cpp > displayArray(int [])

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);

    // std::cout << "\nBubble Sort: \n" << std::endl;
    // bubbleSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    std::cout << "\nSelection Sort: \n" << std::endl;
    selectionSort(unsortedArr, maxSize);
    displayArray(unsortedArr);

    // std::cout << "\nInsertion Sort: \n" << std::endl;
    // insertionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

```cpp
template<typename T>
//array, current postion, size of array
int routineSmallest(T arr[], int K, const int arrSize){
    int position;
    T smallestElem = arr[K];

    for(int j = K + 1; j<arrSize; j++){
        if(arr[j] < smallestElem){
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;

}
template<typename T>
void selectionSort(T arr[], size_t arrSize){
    int POS, temp, pass;
    for(int i=0; i < arrSize; i++){
        POS = routineSmallest(arr, i, arrSize);
        //swapping
        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
        //verification
        pass++;
    }
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 7> cd "c:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 7\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Unsorted Array:

98 43 70 15 84 37 57 74 12 57 40 13 8 23 47 3 10 33 6 59 10 9 86 73 89 58 95 72 8 59 88 54 25 57 0 75 61 34 98 58 81 81 79 10
4 48 98 44 87 4 8 46 20 16 82 8 53 64 8 36 71 83 51 39 67 55 45 62 41 45 79 78 30 64 27 8 37 65 39 63 86 79 26 35 96 20 58 38
8 25 94 53 56 43 64 34 31 98 81 39

Selection Sort:

0 3 4 6 8 8 8 8 8 8 9 10 10 10 12 13 15 16 20 20 23 25 25 26 27 30 31 33 34 34 34 35 36 37 37 38 39 39 39 40 41 43 43 44 45 45
46 47 48 51 53 53 54 55 56 57 57 57 58 58 58 59 59 70 61 62 63 64 64 64 65 67 71 84 72 73 74 75 78 78 79 79 79 98 81 81 81 82
6 83 86 87 88 89 94 95 96 98 98 98
```

Observations	For the third task, I implemented a templated selection sort function in sortingAlgo.h using a helper function, routineSmallest, to locate the smallest element's index in the unsorted portion of the array before swapping it into place. This approach maintains both a best and worst case time complexity of O(n^2). The console output confirmed that the randomly generated array was successfully sorted in ascending order.

Table 7-4. Insertion Sort Algorithm

Code + Console Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize] = {5, 2, 9, 1, 5};
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);

    // std::cout << "\nBubble Sort: \n" << std::endl;
    // bubbleSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    // std::cout << "\nSelection Sort: \n" << std::endl;
    // selectionSort(unsortedArr, maxSize);
    // displayArray(unsortedArr);

    std::cout << "\nInsertion Sort: \n" << std::endl;
    insertionSort(unsortedArr, maxSize);
    displayArray(unsortedArr);
}

void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

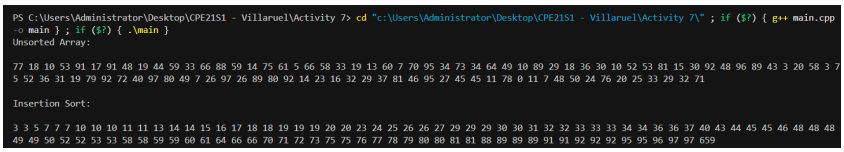
```cpp
template<typename T>
void insertionSort( T arr[], size_t arrSize){
    int K = 0, J, temp;

    while(K < arrSize){
        temp = arr[K];
        J = K - 1;

        while(temp <= arr[J]){
            arr[J + 1] = arr[J];
            J--;
        }

        arr[J + 1] = temp;
        K++;
    }
}

#endif
```

	
Observations	I implemented a templated insert sort function in sortingAlgo.h using nested loops to shift larger elements to the right and insert each key into its correct sorted position. This algorithm has best case time complexity of O(n) when the array is already sorted and a worst case time complexity of O(n^2) when the elements are in reverse order. The console output verified that the randomly generated array was successfully sorted in ascending order.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Pseudocode	START Initialize candidate names array Generate 100 random votes between 1 and 5 Sort votes using selection sort Count votes for each candidate (1 to 5) Determine candidate with highest votes Display sorted values, candidate tallies, and the winner END

Code + Console Screenshot

main.cpp | main.exe | sortingAlgo.h | suppmain.cpp | suppAlgo.h

suppmain.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "suppAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };
    int votes[maxSize];
    srand(time(0));

    //generate random votes
    for (int i = 0; i < maxSize; i++) {
        votes[i] = (rand() % 5) + 1;
    }

    std::cout << "Unsorted votes: \n";
    displayArray(votes);

    //sort votes
    selectionSort(votes, maxSize);
    std::cout << "\nSorted votes: \n";
    displayArray(votes);

    //count and track
    int voteCounts[5] = {0};
    int winnerIndex = 0;

    for(int i = 0; i < maxSize; i++) {
        voteCounts[votes[i] - 1]++;
    }

    std::cout << "\nVote total: \n";
    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << (i + 1) << " (" << candidates[i]
                  << "):  " << voteCounts[i] << " votes\n";

        if (voteCounts[i] > voteCounts[winnerIndex]) {
            winnerIndex = i;
        }
    }
    std::cout << "\nWinning Candidate: " << candidates[winnerIndex]
              << " with " << voteCounts[winnerIndex] << " votes\n";

    return 0;
}
```

```
#ifndef SUPPALGO_H
#define SUPPALGO_H
#include <cstddef>

template <typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position = K;
    T smallestElem = arr[K];

    for(int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize) {
    int POS;
    T temp;

    for (size_t i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);
        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}

#endif
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 7> cd "c:
\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 7\" ; if ($?)
{ g++ suppmain.cpp -o suppmain } ; if ($?) { .\suppmain }
Unsorted votes:
1 5 4 4 5 4 5 4 2 2 3 4 4 2 2 4 1 2 1 1 5 2 3 1 4 4 2 5 1 3 2 2 1 4 3 1
2 5 2 2 2 2 3 4 2 3 3 4 3 3 4 5 5 5 1 2 5 3 1 2 5 1 3 3 1 4 4 3 4 4 3 2
1 3 3 4 4 2 5 1 1 2 3 1 5 5 3 5 1 3 2 2 2 4 2 1 3 4 1 2

Sorted votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote total:
Candidate 1 (Bo Dalton Capistrano):  19 votes
Candidate 2 (Cornelius Raymon Agustin):  25 votes
Candidate 3 (Deja Jayla Banaga):  20 votes
Candidate 4 (Lalla Brielle Yabut):  21 votes
Candidate 5 (Franklin Relano Castro):  15 votes

Winning Candidate: Cornelius Raymon Agustin with 25 votes
```

Question/Justification

Yes, the algorithm was effective because sorting grouped identical votes together, allowing the program to count each candidate's votes sequentially and identify the winner. Selection sort was specifically chosen because it minimizes element swaps to O(n), which is ideal for organizing fixed size data. However, sorting adds unnecessary O(n^2) overhead for vote counting, as a simple frequency array could directly tally the votes in O(n) time without sorting.

Conclusion

For this laboratory activity, I gained a deeper understanding of how basic sorting algorithms operate by implementing bubble sort, selection sort, and insertion sort in C++ to analyze their step by step executions and time complexities. While the concepts and logic felt confusing at first, working through the implementation and applying selection sort to solve the supplementary activity made everything much clearer and easier to do. Ultimately, seeing the sorted data and accurate tallies in the output helped me appreciate how choosing the proper algorithm optimizes data organization and solving real world scenarios.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.